```
      format.json { render :show, status: :ok, location: @concert }
    else
      format.html { render(:edit) }
      format.json {
        render(
          json: @concert.errors,
          status: :unprocessable_entity
        )
      }
    end
  end
end
```

Now the controller actions will call the methods we just added, which will update the data.

The new data is now available to update the display. We can do this in the partial file that draws each individual concert. This key part is this line of code that goes right after the turbo_frame_tag is declared:

chapter_10/05/app/views/concerts/_concert.html.erb

```
<% if current_user.editing?(concert) %>
  <%= render "concerts/form", concert: concert %>
<% else %>
```

The previous contents of this partial (the article tag) go in the else block, the end of this if statement is after the end of that article.

The logic here now works, whenever the app draws a partial concert (which is mostly when the whole page is rendered), it will look at the editing? method, which checks with Kredis and draw the form or the show version based on that value.

It doesn't *quite* work with the code as is. The form partial as written would duplicate the turbo_frame_tag, so you have to change the edit page and the _form partial to move the tag creation to the edit page—the chapter_10/05 directory will show the finished code.

You can verify this by running the app, opening the edit form for a concert, then